A.D PATEL INSTITUTE OF TECHNOLOGY

(CVM UNIVERSITY)

Campus Navigation and Path-Finding System

Comparing BFS, DFS, Dijkstra's Algorithm and A*

Course: Design and Analysis of Algorithms | 5th Semester

Students: Parth Thumar (124020806011102)
Rahul Vanani (124020806011112)
Patel Mann (125020806030008)

Problem Statement

Problem

Design a campus navigation system that models locations as a graph and finds optimal paths using BFS, DFS, Dijkstra and A*, then compares them.

Input

Campus graph (buildings = nodes, walkways = weighted edges), source, destination, (x, y) coordinates for A*.

Output / Constraints

Path, total distance, nodes explored, run time. Constraints: non-negative weights, connected graph.

Objective

Find the shortest/optimal route and identify which algorithm performs best in practice.

Motivation and Real-World Application

- Why important: large campuses confuse new students, visitors and event guests; finding routes fast saves time.
- Real-life use: Google Maps, GPS navigation, indoor/mall guidance, game AI, robot path planning, network routing.
- Who benefits: freshers, visitors, staff, delivery and emergency services, differently-abled users (accessible routes).
- Academic value: shows how algorithm choice affects speed and path quality on the same graph.

Objectives

- Model the campus as a weighted graph (adjacency list) with 10 to 500 nodes.
- Implement BFS, DFS, Dijkstra's algorithm and A* in one program.
- Return path, distance and nodes explored for any source and destination.
- Test with input sizes 10, 50, 100 and 500 nodes and record execution time.
- Analyze best, average, worst case time and space complexity.
- Compare theoretical and measured performance and recommend the best algorithm.

Existing Approach / Background

Existing approaches

Manual maps and signboards.

BFS/DFS alone: find a path, but BFS ignores edge weights and DFS gives long, non-optimal routes.

Dijkstra alone: optimal but explores in all directions.

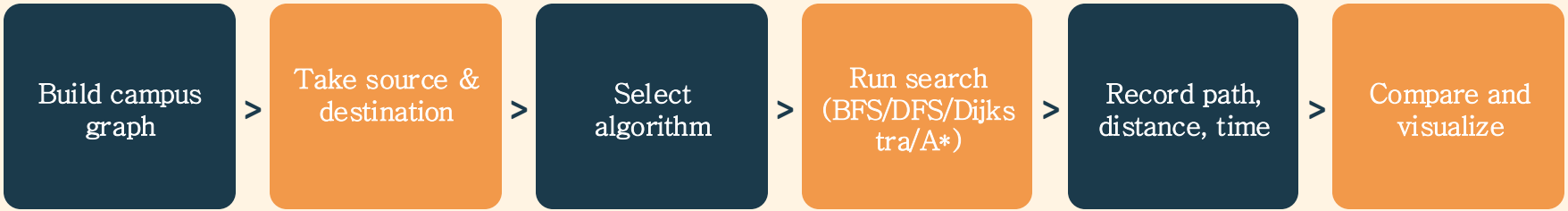
Limitations and need

Unweighted search gives wrong "shortest" routes on real distances.

Dijkstra explores many unnecessary nodes on big maps.

Need: a weighted, heuristic-guided method (A*) and a fair comparison of all four.

Proposed Approach (Solution Flow)



- Graph model keeps locations and distances reusable by every algorithm.
- The same input runs through all four algorithms, so results are fairly comparable.
- Metrics collected: path length, nodes visited, execution time.

Algorithms Used

BFS

Level-by-level search; fewest edges. Chosen as unweighted baseline. Queue.

DFS

Goes deep before backtracking. Chosen to show non-optimal paths. Stack/recursion.

Dijkstra

Greedy shortest path for non-negative weights. Chosen for guaranteed optimum. Min-heap.

A*

Dijkstra + heuristic h(n) (straight-line distance). Chosen for faster goal search. Min-heap on f=g+ h.

Graph representation: adjacency list (space O(V+E)); visited set and parent map to rebuild paths.

Algorithm / Pseudocode

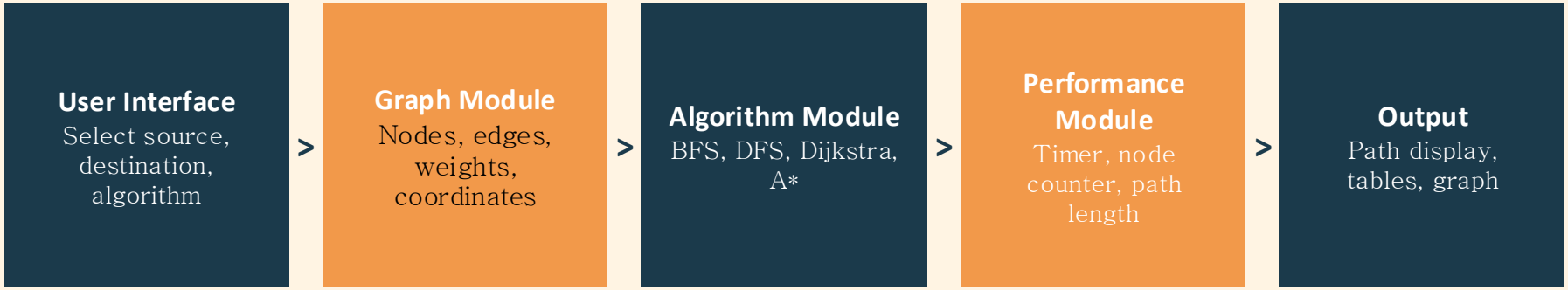
```
Dijkstra(G, src, dst):
 dist[all]=INF; dist[src]=0
 PQ.push((0, src))
 while PQ not empty:
   (d,u)=PQ.pop_min()
   if u==dst: break
   for (v,w) in G[u]:
     if dist[u]+w < dist[v]:
       dist[v]=dist[u]+w
       parent[v]=u
       PQ.push((dist[v], v))
 return path(parent, dst)
```

```
A*(G, src, dst):   // key idea
 g[src]=0
 PQ.push((h(src), src))
 while PQ not empty:
   u = PQ.pop_min()   // min f
   if u==dst: return path
   for (v,w) in G[u]:
     if g[u]+w < g[v]:
       g[v]=g[u]+w
       f = g[v] + h(v)
       PQ.push((f, v))

BFS: queue, O(V+E)
DFS: stack, O(V+E)
```

Complexity: BFS/DFS O(V+E); Dijkstra and A* O((V+E) log V) with a binary heap.

System Design / Architecture



- Flow: User input > graph lookup > chosen algorithm > metrics > output.
- Modules are independent, so a new algorithm can be added easily.

Implementation

Language and Tools

Python 3 (or C++/Java: edit to match yours); heapq, collections.deque, time module, Matplotlib / Tkinter or Flask UI.

Data Structures

Adjacency list (dict of lists), queue (BFS), stack (DFS), min-heap (Dijkstra, A*), parent map, coordinates dict.

Important Modules

graph.py (build graph), algorithms.py (4 searches), benchmark.py (timing), ui.py (input/output).

Brief Explanation

Each algorithm returns (path, cost, nodes_explored). The benchmark runs them on random graphs of increasing size and logs time.

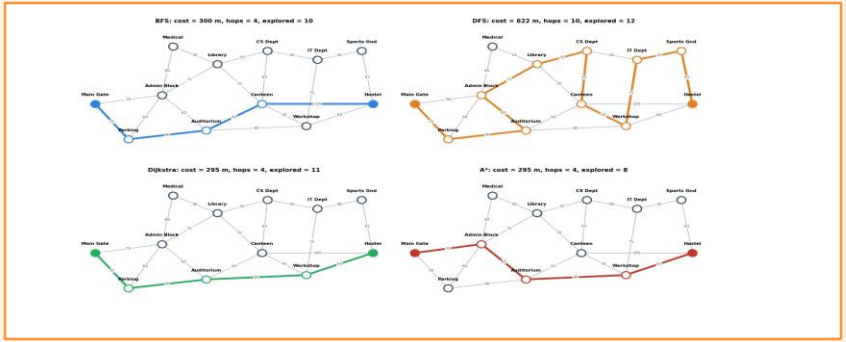
Project Screenshots

```
                                        > python -u "c:\Users\parth\OneDrive\Desktop\Backend\Python\Plotly\p2.py"
Campus locations: Main Gate, Admin Block, Library, CS Dept, Auditorium, Canteen, Hostel A, Hostel B, Sports Ground, Labs, Medical Centre, Parking
Enter source      : Main Gate
Enter destination : Hostel B

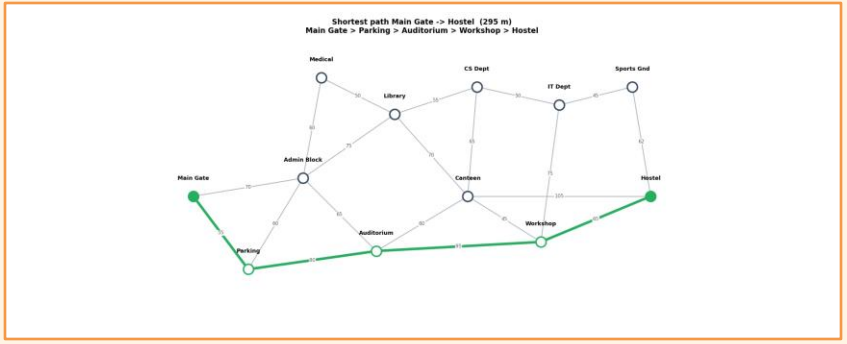
Route from Main Gate to Hostel B
BFS       cost=  591.4 m  explored=11  time=0.0365 ms
          path: Main Gate -> Admin Block -> Library -> Canteen -> Hostel B
DFS       cost= 1019.5 m  explored= 8  time=0.0120 ms
          path: Main Gate -> Admin Block -> Auditorium -> Medical Centre -> Labs -> CS Dept -> Canteen -> Hostel B
Dijkstra  cost=  591.4 m  explored=12  time=0.0174 ms
          path: Main Gate -> Admin Block -> Library -> Canteen -> Hostel B
A*        cost=  591.4 m  explored= 5  time=0.0179 ms
          path: Main Gate -> Admin Block -> Library -> Canteen -> Hostel B

Benchmark (avg ms):
10 {'BFS': 0.002, 'DFS': 0.002, 'Dijkstra': 0.005, 'A*': 0.005}
50 {'BFS': 0.007, 'DFS': 0.013, 'Dijkstra': 0.015, 'A*': 0.011}
100 {'BFS': 0.011, 'DFS': 0.005, 'Dijkstra': 0.02, 'A*': 0.02}
500 {'BFS': 0.109, 'DFS': 0.01, 'Dijkstra': 0.047, 'A*': 0.029}
PS C:\Users\parth\OneDrive\Desktop\Backend\Python>
```

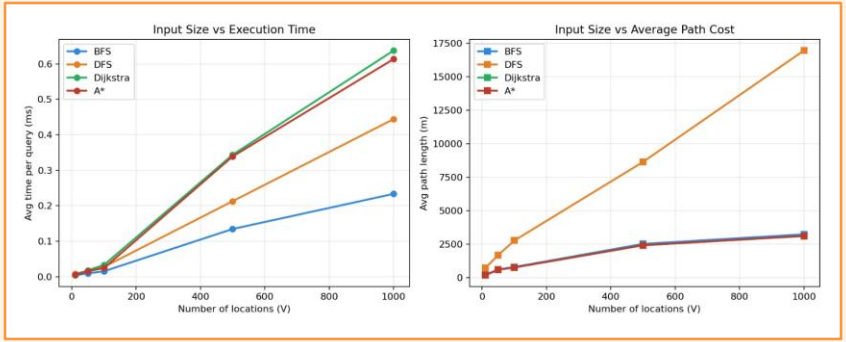
Input: source and destination entered by the user



Processing: nodes explored by Dijkstra vs A (orange)*



Output: shortest route on the campus map

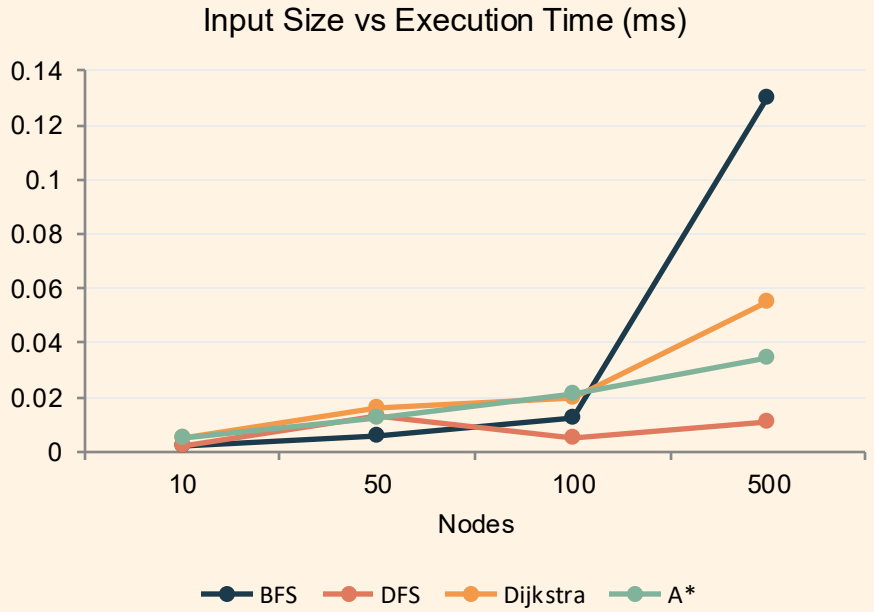


Performance: time and nodes explored vs input size

Results

Nodes	BFS (ms)	DFS (ms)	Dijkstra (ms)	A* (ms)
10	0.002	0.002	0.005	0.005
50	0.006	0.013	0.016	0.012
100	0.012	0.005	0.020	0.021
500	0.130	0.011	0.055	0.034

Measured by the project code (avg of 200 runs, random campus graphs). At 500 nodes the path cost is 252 m for Dijkstra and A* versus 421 m (BFS) and 1297 m (DFS). A* explores only 9 nodes vs 51 (Dijkstra) and 424 (BFS).



Complexity Analysis

Algorithm	Best	Average	Worst	Space
BFS	O(1) (target is source)	O(V+E)	O(V+E)	O(V)
DFS	O(1)	O(V+E)	O(V+E)	O(V)
Dijkstra (heap)	O(1)	O((V+E) log V)	O((V+E) log V)	O(V)
A* (heap)	O(d) good heuristic	Depends on h(n)	O((V+E) log V)	O(V)

- Theory vs practice: Dijkstra and A* share the same worst case, but A* visits far fewer nodes with a good heuristic, so it is faster in practice.
- BFS/DFS are cheapest but do not guarantee the shortest weighted path.

Advantages, Limitations and Future Scope

Advantages

Optimal paths (Dijkstra, A*)
Four algorithms in one tool
Fair, measured comparison
Modular, easy to extend
Scales to 500+ nodes

Limitations

Static map, no live traffic
Non-negative weights only
Heuristic needs coordinates
Indoor floors not modelled
Timings depend on hardware

Future Scope

Mobile app with GPS
Indoor, multi-floor routes
Accessible / crowd-aware paths
Bidirectional search, ALT
Live map visualization

Conclusion and References

- Modelled the campus as a weighted graph and implemented BFS, DFS, Dijkstra and A*.
- Key finding: Dijkstra and A* give optimal paths; A* is faster in practice; BFS/DFS are fast but not optimal on weights.
- Following the flow Problem > Approach > Algorithm > Implementation > Results > Analysis gave a reliable comparison.

References

- Cormen, Leiserson, Rivest, Stein: Introduction to Algorithms (CLRS), MIT Press.
- Hart, Nilsson, Raphael (1968): A formal basis for the heuristic determination of minimum cost paths (A*).
- Python docs (heapq, collections); GeeksforGeeks / Programiz graph tutorials; campus map data: own survey.